GUJARAT TECHNOLOGICAL UNIVERSITY

Project Based Learning (PBL) Report

Market Basket Analysis

A Real-World Case Study using the Apriori Algorithm

What do electronics customers buy together?

PROJECT DETAILS

Project Title	Market Basket Analysis: A Real-World Case Study
Subject	Project Based Learning (PBL)
Technique Used	Apriori Association Rule Mining (Python, pandas, mlxtend)
Student Name	Ghoghara Pujan Hareshkumar
Enrollment Number	240280152023
Team Members	Dhruv Pandav Abhiprakash
Branch / Semester	AIML/5
Institute / College	L.D. College of Engineering
Guide / Faculty	Prachi Pancholi
Academic Year	2026
Submission Date	15 October 2026

Section	Page

Abstract	3
1. Introduction	3
2. Objectives	3
3. Concepts Used	4
4. Dataset Description	4
5. Methodology	5
6. Implementation and Code Explanation	5
7. Results	7
8. Discussion of Findings	9
9. Business Recommendations	9
10. Limitations	9
11. Future Scope	10
12. Conclusion	10
13. References	10

Market basket analysis is a data mining technique that discovers which products customers tend to buy together. This report applies the Apriori algorithm to a dataset of 500 electronics orders containing 10 distinct products. The raw order data was inspected, cleaned, converted into a transaction matrix and mined for frequent itemsets and association rules using Python (pandas and mlxtend).

At a minimum support of 5%, only 2 rules were found, so the threshold was lowered to 2%. This produced **77 frequent itemsets** (10 single items, 45 pairs and 22 triples) and a much richer set of rules. The strongest rule was **iStream + pearTV → Alcon 10** with a lift of **1.52** and a confidence of **41.4%**. The findings are translated into recommendations for bundling, product recommendations and promotions, and the limitations of the results are discussed honestly.

500	10	77	1.52
orders	products	frequent itemsets	highest lift

1. Introduction

Retailers collect large amounts of transaction data, but the data is only valuable if it is converted into decisions. Market basket analysis answers a simple question: *which items appear together in the same order more often than chance would suggest?* The classic example is a supermarket discovering that customers who buy bread often buy butter.

The same idea applies to an online electronics store, where it supports product bundles, "customers also bought" recommendations, cross-selling and promotion design. This case study uses order-level data from an electronics retailer and applies association rule mining to find product combinations that sell together.

2. Objectives

- Understand and clean the raw order data and extract the products in each shopping cart.
- Convert the data into a transaction format suitable for association rule mining.
- Find frequent itemsets and association rules using the Apriori algorithm.
- Evaluate rules using support, confidence and lift.
- Translate the rules into business recommendations and note their limitations.

3.1 Association rules

An association rule has the form **A → B**, meaning that orders containing the items in A (the antecedent) also tend to contain the items in B (the consequent).

3.2 Key metrics

Metric	Meaning	Formula
Support	How often the itemset appears across all orders	count(A and B) / total orders
Confidence	Of the orders containing A, the share that also contain B	support(A and B) / support(A)
Lift	How much more often A and B occur together than if they were independent	confidence / support(B)

A lift above 1 means the items appear together more than chance, a lift of 1 means no relationship, and a lift below 1 means the items tend not to appear together.

3.3 The Apriori algorithm

Apriori finds frequent itemsets level by level. It first finds all single items that meet the minimum support, then builds pairs only from frequent single items, then triples only from frequent pairs, and so on. It relies on the Apriori property: *if an itemset is infrequent, every larger itemset containing it must also be infrequent*. This lets the algorithm skip huge numbers of combinations. Rules are then generated from the frequent itemsets and filtered by confidence and lift.

4. Dataset Description

The dataset (dirty_data.csv) contains 500 rows and 16 columns, including order_id, customer_id, date, nearest_warehouse, shopping_cart, order_price, delivery_charges, coupon_discount, order_total, season, is_expedited_delivery, distance_to_nearest_warehouse, latest_customer_review and is_happy_customer.

Fact	Value
Orders	500 (all unique, 0 duplicate rows)
Unique customers	494
Distinct products	10
Average items per order	3.016

Basket size range	2 to 4 products
Missing values	1 (in latest_customer_review)

The 10 products are: Lucent 330S, Thunder line, Candle Inferno, Olivia x460, Universe Note, pearTV, Toshika 750, iStream, Alcon 10 and iAssist Line.

The key field, shopping_cart, was stored as text (for example `[('Thunder line', 1), ('pearTV', 2)]`), so it had to be parsed before analysis. Each tuple holds a product name and a quantity. Data

quality was otherwise good, except that product names appear in 29 different capitalisation spellings (see Section 6.4).

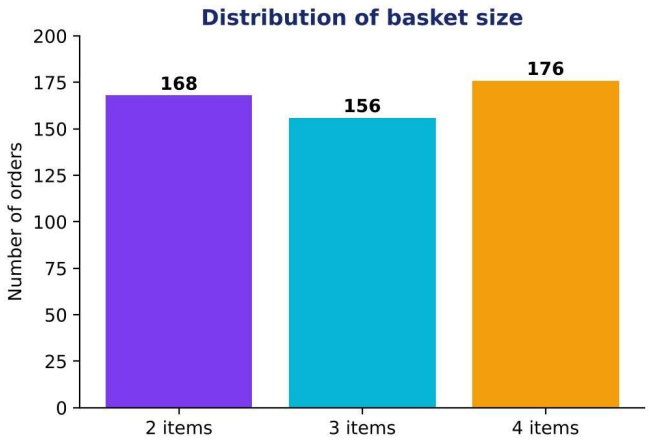


Figure 1: Basket size distribution (168 orders with 2 items, 156 with 3 items, 176 with 4 items)

5. Methodology

The work followed a five-step workflow.

1 Inspect	2 Parse	3 Encode	4 Mine	5 Rules
Shape, types, missing values, duplicates	Convert shopping_cart text into product lists	Order x product True/False matrix	Frequent itemsets at 5% then 2% support	Keep lift > 1.1, confidence >= 30%

6. Implementation and Code Explanation

The analysis was implemented in Python using Google Colab with the libraries pandas, ast, matplotlib and mlxtend.

6.1 Loading and inspecting the data

```
df = pd.read_csv('/content/dirty_data.csv')
print(df.shape); print(df.dtypes)
print(df.isnull().sum()); print(df.duplicated().sum())
```

`pd.read_csv` loads the file. The shape, data types, missing-value count and duplicate count confirm the structure, the single missing review and the absence of duplicates. `nunique()` on order_id and customer_id gives 500 orders from 494 customers.

6.2 Parsing the shopping cart

```
import ast df['cart'] =
df['shopping_cart'].apply(ast.literal_eval)
df['basket_size'] = df['cart'].apply(len)
```

The cart column is a string that looks like a list. `ast.literal_eval` safely converts it into a real list of (product, quantity) tuples. The length of each list gives the basket size: average 3.016, range 2 to 4.

6.3 Extracting product names

```
df['products'] = df['cart'].apply(
lambda x: [item[0] for item in x])
```

Association rules care only whether a product is present, not its quantity, so a list comprehension keeps just the product name. For example, order ORD182494 becomes ['Lucent 330S', 'Thunder line', 'iStream', 'pearTV'].

6.4 One-hot encoding

```
from mlxtend.preprocessing import TransactionEncoder te =
TransactionEncoder() te_array =
te.fit(df['products']).transform(df['products']) basket =
pd.DataFrame(te_array, columns=te.columns_)
```

`TransactionEncoder` converts the product lists into a Boolean matrix with one row per order and one column per product. A cell is True if that product was in the order. This is the input format Apriori requires. The raw product names are written in 29 different spellings (for example "Alcon 10", "ALCON 10" and "alcon 10"), so the matrix has **29 columns** (shape 500 x 29) instead of 10. Only the correctly written names reach the support thresholds, so the 10 products carry all results below. Merging the spellings is possible but gives slightly different numbers (for example the top lift becomes 1.42), so the results here follow the original, uncleaned data.

6.5 Frequent itemsets with Apriori

```
from mlxtend.frequent_patterns import apriori
frequent_itemsets = apriori(basket, min_support=0.05,
use_colnames=True)
```

This returns every itemset appearing in at least 5% of orders. At 5% it found 53 itemsets. A column `itemset_size` was added to separate singles, pairs and triples.

6.6 Generating rules

```
from mlxtend.frequent_patterns import association_rules
rules = association_rules(frequent_itemsets,
                          metric='lift',
min_threshold=1.0)
```

Rules are created from the itemsets and kept if lift is at least 1.0. At 5% support only **2 rules** appeared (pearTV → Alcon 10 and the reverse, both lift 1.16), which is too few to be useful.

6.7 Lowering the support threshold

```
frequent_itemsets_02 = apriori(basket, min_support=0.02,
use_colnames=True)
```

Because the products are evenly popular and baskets are small, specific combinations are rare. Lowering min_support to 2% produced **77 frequent itemsets**: 10 single items, 45 pairs and 22 triples.

6.8 Selecting useful rules

```
selected_rules = rules_02[
(rules_02['support'] >= 0.02) &
    (rules_02['confidence'] >= 0.30) &
(rules_02['lift'] > 1.10)].copy()
selected_rules = selected_rules.sort_values(
by=['lift','confidence'], ascending=False)
```

Rules were filtered with three conditions: support at least 0.02, confidence at least 0.30 and lift above 1.10, then sorted by lift. The frozensets were converted into readable text such as "iStream, pearTV → Alcon 10", and support and confidence were converted to percentages. A bar chart of the top 8 rules by lift was then plotted with matplotlib.

7. Results

7.1 Product popularity

Demand is spread evenly: each product appears in about 26% to 32% of orders. Lucent 330S is the most frequent at 31.6%, and iAssist Line the least at 26.0%. There is no single blockbuster product.

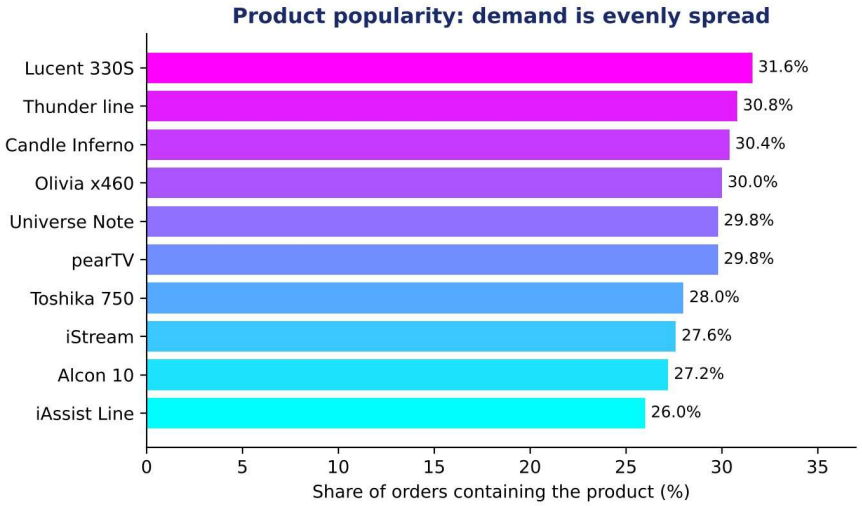


Figure 2: Share of orders containing each product

7.2 Pairs of products

The strongest pair is pearTV with Alcon 10, appearing together in 9.4% of orders. Its lift is only 1.16, so the association is weak. Other common pairs such as Universe Note with Thunder line (9.0%) are also close to what chance would produce.

7.3 Top association rules

Rule	Support (%)	Confidence (%)	Lift
iStream, pearTV → Alcon 10	2.4	41.38	1.52
Alcon 10, Candle Inferno → Toshika 750	2.6	37.14	1.33
Alcon 10, Thunder line → Toshika 750	2.0	37.04	1.32
pearTV, Universe Note → Thunder line	3.2	38.10	1.24
pearTV, iAssist Line → Alcon 10	2.2	33.33	1.23
pearTV, Thunder line → Universe Note	3.2	36.36	1.22
Toshika 750, Candle Inferno → Alcon 10	2.6	32.50	1.19
Thunder line, Universe Note → pearTV	3.2	35.56	1.19
Lucent 330S, pearTV → Alcon 10	2.6	31.71	1.17
Alcon 10 → pearTV	9.4	34.56	1.16

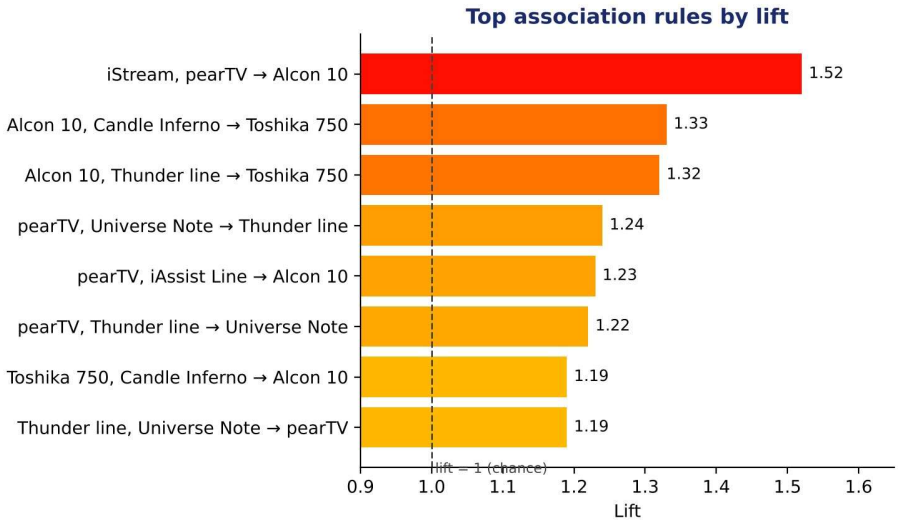


Figure 3: Top 8 association rules ranked by lift (dashed line = no association)

7.4 How to read the top rule

The rule **iStream + pearTV → Alcon 10** has a lift of 1.52. About 41% of orders containing both iStream and pearTV also contain Alcon 10, compared with 27.2% of all orders. Alcon 10 is therefore about 1.5 times more likely in those orders than in a typical one. Similarly, orders with Alcon 10 plus Candle Inferno or Thunder line include Toshika 750 about 37% of the time, against 28% overall.

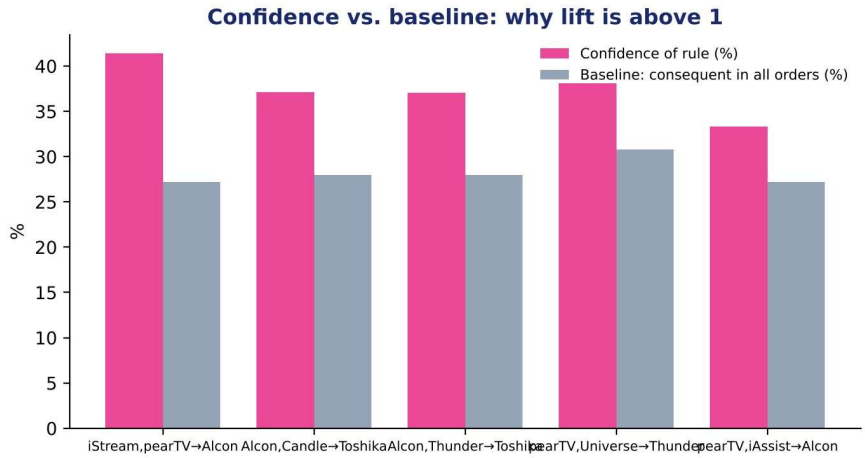


Figure 4: Rule confidence compared with the baseline share of the predicted product

8. Discussion of Findings

- **No single star product.** All 10 products have similar demand, so simple best-seller lists reveal little. The combinations carry the insight.
- **Pairs are weak, triples are stronger.** The best associations appear when two products together predict a third.
- **Alcon 10 is the anchor product.** It appears in most of the top rules, as the predicted item or as part of the antecedent.

- **pearTV is a frequent partner.** It appears in many strong rules, making it a good lead item for promotions.

9. Business Recommendations

#	Recommendation	Why
1	Bundle iStream + pearTV with Alcon 10	Highest-lift rule (1.52, confidence 41.4%)
2	Suggest Toshika 750 when Alcon 10 and Candle Inferno or Thunder line are in the cart	About 37% of such orders already include it (lift 1.32 to 1.33)
3	Use Alcon 10 and pearTV as anchor products in promotions	They sit at the centre of many rules
4	Show rules as "customers also bought" prompts, not guarantees	Associations are moderate, not strong

10. Limitations

- **Low coverage.** The strong rules cover only about 2% to 3% of orders, roughly 10 to 16 orders out of 500, so a small change in the data could shift the results.
- **Moderate lift.** Lift values of 1.2 to 1.5 indicate a moderate association, not a strong one.
- **Small dataset.** With 500 orders and 10 products, the findings may not generalise to a full retail catalogue.
- **Quantity ignored.** Only presence or absence of a product is analysed, not units bought.
- **Inconsistent product names.** The 10 products appear in 29 spellings. Entries with unusual capitalisation were not merged, so some purchases are not counted and support values are slightly understated.
- **Association is not causation.** A rule shows items co-occur, not that one purchase causes the other.
- **Lowered threshold.** Moving from 5% to 2% support was necessary to find rules but raises the chance of coincidental patterns.

11. Future Scope

- Test the rules on a larger dataset or a held-out time period to check that they hold.
- Compare Apriori with FP-Growth, which is faster on large data.
- Add other signals such as season, coupon use and customer location to find segment-specific rules.
- Run an A/B test of the recommended bundles to measure real sales impact.

12. Conclusion

This case study showed how association rule mining turns raw order records into actionable retail insight. After parsing and encoding 500 electronics orders, Apriori at 2% support revealed 77 frequent itemsets and a set of rules in which Alcon 10 and pearTV act as anchor products. The strongest rule, **iStream + pearTV → Alcon 10**, has a lift of 1.52 and a confidence of 41.4%.

Although the associations are moderate and cover a small share of orders, they give the business clear starting points for bundles, recommendations and promotions, and a sound basis for further testing on larger data.

13. References

- Agrawal, R. and Srikant, R. (1994). Fast Algorithms for Mining Association Rules. Proceedings of the 20th VLDB Conference.
- Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. Journal of Open Source Software.
- mlxtend documentation: apriori and association_rules.
- pandas documentation: https://pandas.pydata.org
- Han, J., Kamber, M. and Pei, J. Data Mining: Concepts and Techniques.